第3章 Python语言快速入门

章节导航

- 了解Python语言的起源和应用领域。
- 了解Python语言的特点，掌握Python语言的基础语法，Python语言与其他编程语言的特性差异。
- 能搭建Python环境，学会在命令行、PyCharm编辑器和Jupyter Notebook等环境下编写Python代码并执行。

本章知识导图

3.1 Python语言基本简介

本节主要介绍Python语言的起源、特性和集成开发环境，目的是让刚步入数据科学领域的读者了解Python语言的应用领域，理解使用Python语言进行数据科学分析与进行软件开发的区别。

3.1.1 Python语言的历史及学习路线

Python语言是一种通用的解释型高级语言，由吉多·范罗苏姆(Guido van Rossum)于1991年发明。它具有灵活、高效、易学的特点，并包含一组功能完备的标准库，用户使用Python语言可以轻松地完成许多常见任务。

在Python语言出现之前，人们通常使用C/C++语言编写程序，C/C++语言是一种编译型语言，需要先使用编译器编译程序，然后使用解释器执行程序。相比之下，Python语言是一种解释型语言，程序按行执行，省去了编译的步骤，使得程序的运行更加简洁。然而，与C/C++语言相比，Python语言的运行速度相对较慢。

Python语言的每种数据类型都是面向对象的，并进行了良好的封装。在定义函数和命名变量时，Python语言将这些内容都隐藏起来，使其对初学者更加友好。

Python语言本身是一门相对古老的语言，经典版本有Python 2.X和Python 3.X。考虑到目前大多数数据科学工具是基于Python 3.7或更高版本构建的，建议读者使用Python 3.6～3.8版本学习。如果读者之前已经掌握了Python 2.X的使用，那么可以直接使用Python 3.X，因为Python 2.X和Python 3.X之间只有一些细小的功能差异。Python 3.X有许多版本，截至2024年6月6日，最新版本是3.12.4。

根据学习周期，学习者利用Python语言可以完成以下类型的任务。

(1)初学期(0～3 个月的学习周期)。对于零基础的初学者，通常需要 1～4 周的时间掌握 Python 语言基础语法。如果初学者已经具备 Java 语言或 C 语言的基础，那么可以直接开始接触项目。刚开始接触 Python 语言时，初学者可能会对一些概念感到困惑，但不用担心，随着学习的深入，这些概念会逐渐变得明朗。在这个阶段，可以进行一些简单的文档查找，完成初级的网络爬虫任务等。在学习过程中，除了学习 Python 语言，学习者还会接触到一些操作系统和数据结构的知识，并逐渐掌握一些 HTML、串联样式表(cascading style sheets，CSS)，甚至 Java Script(JS)语言的知识。

(2)进阶期(3～6 个月的学习周期)。在这个阶段，学习者已经开始应用 Python 语言的一些进阶功能，如网络编程、多线程等技术。开始接触一些数据科学任务或系统运维任务，能够理解数据科学项目中的大部分代码，并能够实现报表自动生成和完成较为复杂的爬虫任务。

(3)熟练期(6 个月以上的学习周期)。在这个阶段，学习者已经掌握了大部分 Python 语言的特性，熟练掌握面向对象程序设计的技术和原理，几乎可以用 Python 语言完成任何任务，如 Web 搭建、大数据分析、人脸检测、推荐系统设计、自动化测试等。

大部分人的学习时间是不连续的，可能会因为工作或升学而延长学习周期。请尽量每天花一些时间来学习这门语言，以保持对这门编程语言的“语感”。

使用 Python 语言超过 6 个月的学习者，可以开始在代码开源仓库 GitHub 上搜索一些热门的基于 Python 语言的实践应用。例如，卡耐基梅隆大学的 OpenPose 库，可用于实时检测人体姿态；Haystack 库，可用于构建智能问答应用。

3.1.2　Python 语言的特性

编译型与解释型语言

Python 语言是一门解释型语言。解释型语言更好上手，对于有编程基础的人来说，会觉得 Python 语言十分容易理解。Python 语言使用缩进规范代码块，下面是一个打开文档并读取内容的例子。

```
# 读取 data.txt 文件中的内容并打印
with open("data.txt", 'r', encoding= 'utf-8') as f:
    # Python 语言使用缩进来标识代码块
    content=f.readlines( )
# 将读取的内容打印出来
print(content)
```

Python 语言拥有十分强大的第三方生态库，能够极大程度地扩展 Python 语言的应用范围。例如，通常居民区附近有生鲜超市、公园、医院、地铁、学校等设施，这意味着这个小区的居民生活便利，在程序的世界中，生态也十分重要，使用 Python 语言和一些第三方库，可以很方便地处理图像或视频，将数据持久化存储到 MySQL、SQL Server 数据库中，快速地操作 Excel 文档，使用 wordcloud 第三方库生成漂亮的词云图等。

目前，大多数的人工智能研究使用 Python 语言实现，例如，深度学习算法框架 TensorFlow 和 PyTorch，基于 Python 语言构建的深度学习模型简单易用。此外，基于 Python 语言构建的应用可以使用 Flask 或 Django 等 Web 服务框架实现接口，供其他应用

调用，这种调用也成为一种微服务方式，使得 Python 语言融入 Java、C♯等语言开发的应用。如下代码所示，使用 Python 语言可以很方便地实现深度学习算法的模型创建。

```
from timm.models import create_model

model_name="pretrain_mae_base_patch16_224"
device_name="cuda: 0"
drop_path_rate=0.0
input_size=224
model_path="./model/pretrain_mae_vit_base_mask_0.75_400e.pth"
model=create_model(
    model_name=model_name,
    pretrained=False,
    drop_path_rate=drop_path_rate,
    drop_block_rate=None
)
```

凡事有利必有弊，解释型语言的执行效率低于编译型语言(C＋＋语言、Java 语言都属于编译型语言)，在一些高性能处理场景下，Python 语言可能无法承担重任，但是，计算机的执行速度是很快的，使用 Python 语言搭建一个 Web 服务系统、人脸检测模块或与可编程逻辑控制器(programmable logic controller，PLC)连接提供图像检测功能，系统的响应速度也很快，对于刚入门数据科学的学习者来说，不用担心 Python 语言的执行效率。

3.1.3 常用的集成开发环境

常用的 Python 语言开发工具是 PyCharm 和 Jupyter Notebook。PyCharm 是一款功能强大、易于使用的 Python 语言开发工具，适用于各种规模的项目和开发需求。无论是初学者还是专业开发人员，都可以通过 PyCharm 提高开发效率和代码质量。而 Jupyter Notebook 是一种开源的交互式计算环境，可以在 Web 浏览器中创建和共享文档，其中包含实时代码、数学方程、可视化和说明文本。这两个工具各有优点，PyCharm 适合做工程性质的开发，有良好的代码补全、代码版本控制、项目视图工具，基于 Python 语言的 Web 项目、运维测试、复杂模型搭建脚本，需要使用 PyCharm。Jupyter Notebook 更加轻便，更适合用于数据分析挖掘，数据分析的过程都会展示在 Jupyter Notebook 中，十分方便，但它对复杂工程的支持不是很好。著名的数据科学竞赛网站 Kaggle 就以 Jupyter Notebook 为内容，方便社区用户分享自己的竞赛编码经验。感兴趣的读者请访问 www.kaggle.com。

Jupyter Notebook 介绍

这里简单讲解一下环境搭建。如果操作系统是 Windows，需要用户自行安装 Python 开发环境。Linux 操作系统和 macOS 操作系统自带 Python 开发环境，其版本一般为 Python 2，较新的 Linux 操作系统版本会自带 Python 3，如 Ubuntu 操作系统，版本号为 18.04 或者 20.04 以上，Python 语言的版本为 3.5、3.6 或者 3.7。

此外，可以通过安装一个 Anaconda 软件，降低环境搭建的难度。Anaconda 是专注于

数据分析的 Python 语言发行版本，包含 SciPy、NumPy 等 190 多个科学包及其依赖项。conda 是开源包(packages)和虚拟环境(virtual environment)的管理系统。可以直接在官网下载 Anaconda 软件。

Anaconda Navigator 是 Anaconda 发行版中包含的桌面图形用户界面，是一个导航窗口，如图 3-1 所示，可以使用它启动 Jupyter Notebook 或者 JupyterLab，也可以使用它创建新的 Python 环境或安装新的第三方库。

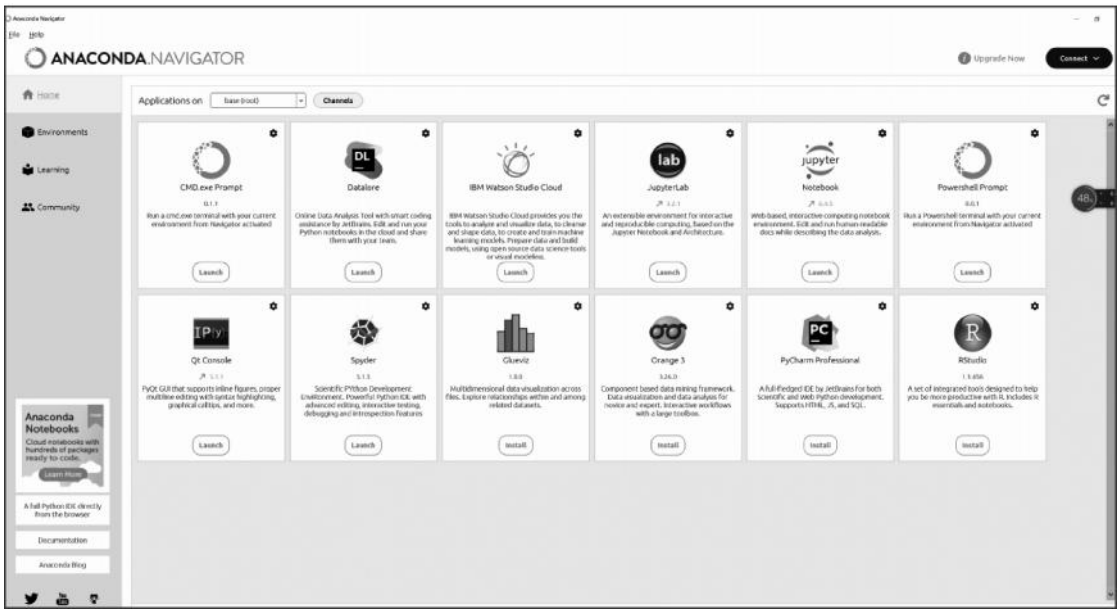

图 3-1　Anaconda Navigator 窗口

Anaconda Prompt 是包含 Anaconda 的一个数据科学计算环境的命令行终端。在这里可以运行操作系统的文件系统，可以在启动栏输入 Anaconda Prompt 启动命令行终端窗口，并使用 pip 命令安装 Jupyter Notebook 工具。安装完成后，输入 Jupyter Notebook，将进入基于 Web 的开发窗口。安装 Jupyter Notebook 的过程如图 3-2 所示。

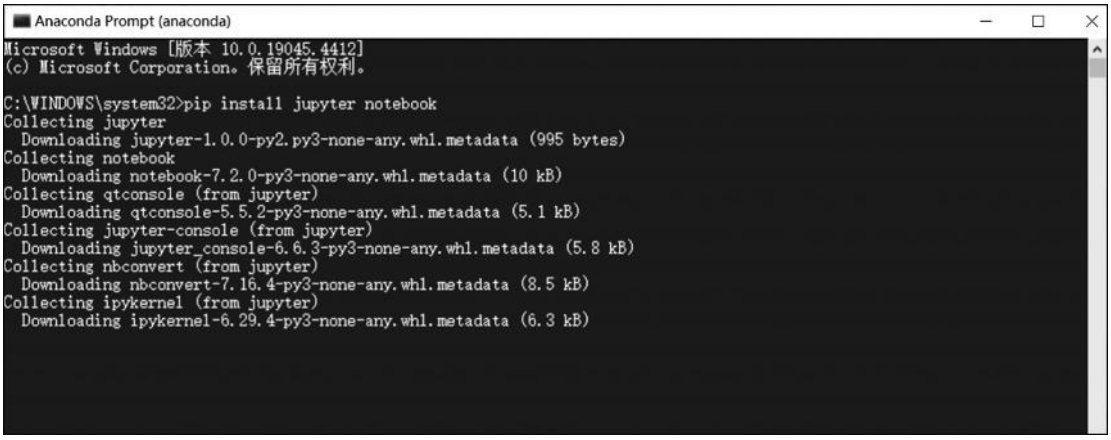

图 3-2　安装 Jupyter Notebook 的过程

Jupyter Notebook(以下简称 Notebook)的使用相对 PyCharm 来说更简单，常用于数据的探索与可视化分析，下面介绍 Notebook 的启动与使用方法。首先在 Anaconda Prompt 中启动 Notebook，程序会在默认的浏览器中打开 Notebook，可以单击窗口右上角的 New 按钮创建一个新的 Notebook，如图 3-3 所示，最上方的菜单栏用于保存文件、修改文件、切换是否启动行号等。窗口中每一个灰色的方块称为一个代码单元(cell)，每个代码单元可以单独运行，也就是说，可以跳过某个代码单元直接运行下一个。需要注意的是，代码单元之间的变量是相互共享的，例如，在第一个代码单元导入了第三方工具库 Pandas，后续的代码单元都不再需要导入 Pandas 就可以直接使用。此外，Notebook 运行文件的后缀是 .ipynb，这与 Python 程序的 .py 格式不同，.ipynb 文件是无法使用 Python 程序直接运行的。

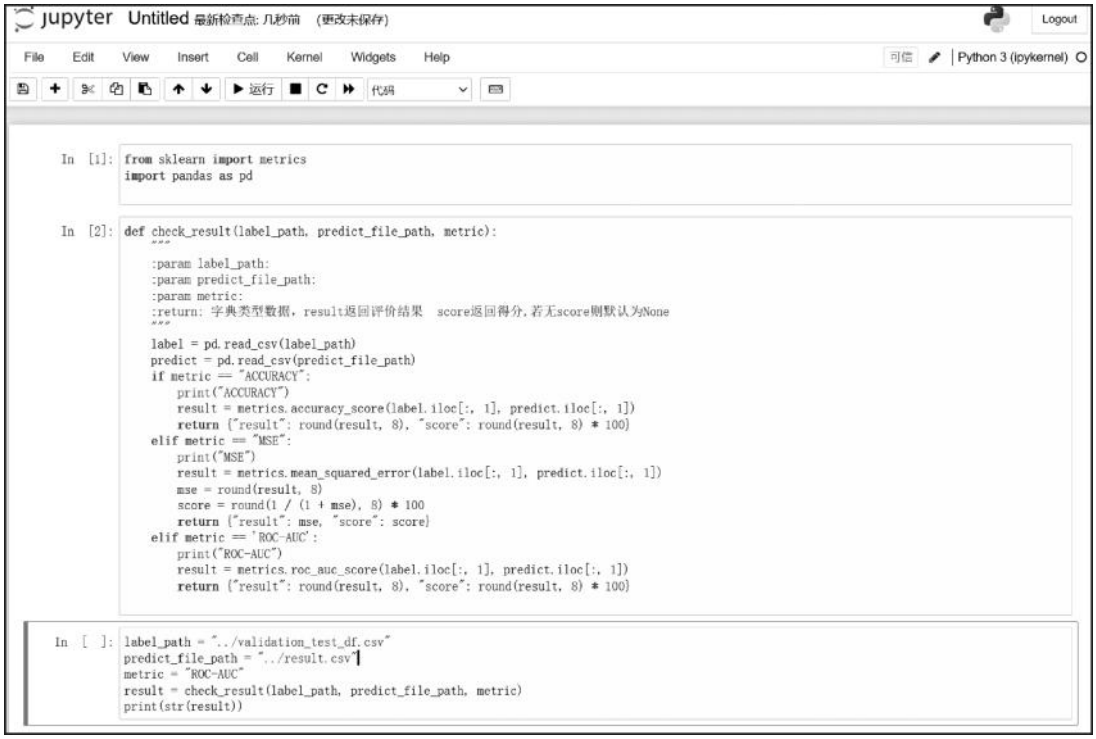

图 3-3　创建一个新的 Notebook 窗口

PyCharm 适合开发一些工程脚本。PyCharm 的功能相对 Jupyter Notebook 来说更多，能够支持代码的跳转、版本管理、编译器配置等。因此它是一款集成开发环境(integrated development environment，IDE)。此外，如果读者是学生，可以登录 JetBrains 官网申请 individual licences for students and teachers 权限，免费使用 JetBrains 开发的所有集成开发环境的 Professional(专业)版本。图 3-4 是 PyCharm 工作窗口。其中，窗口正中央是编辑代码的区域，左侧是工程文件目录，可以调整窗口中央显示的代码文件，上方是菜单栏，单击右上方工具栏的按钮可以运行与调试代码。

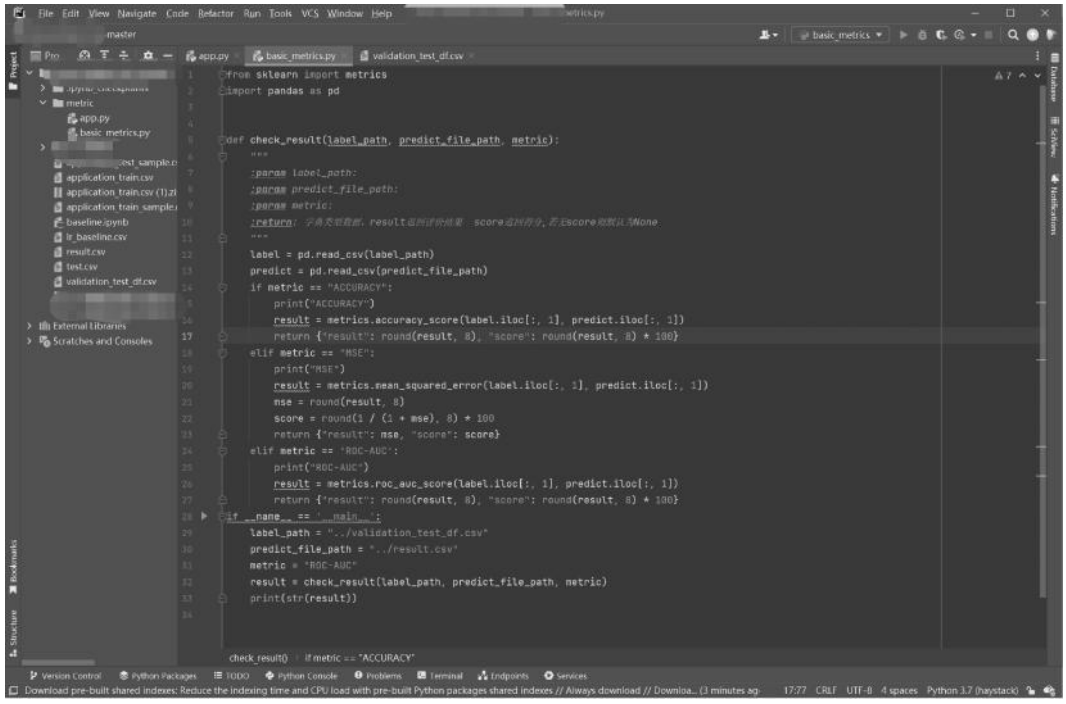

图 3-4　PyCharm 工作窗口

3.2 《Python 语言的语法

本节介绍 Python 语言的语法。Python 语言的基础语法与 JS、C 语言等有很多共同之处，如数据类型、基本操作运算符、函数的定义等。通过本节的学习，读者将掌握基础的 Python 语法，并能够使用 Python 语言完成基础的文件读取与保存操作。

3.2.1　变量及其赋值

1. 变量赋值

和其他语言一样，Python 语言也使用等号(＝)为变量赋值，变量名在等号左侧，需要赋值的内容在等号右侧。关于变量赋值，要注意如下几点：

(1)变量需声明再使用；

(2)不规定变量的类型；

(3)不规定数组长度。

下面代码的功能是使用 Python 语言声明变量。

```
# 此处没有显式声明创建的变量的类型
a=1
b="How Are You"
# 数组长度可变，在 Python 语言中下面的数组也称为列表
c=[1, 2, 3]
```

Java 语言(或 C 语言)是较为严格的编程语言，在声明变量时，需要声明变量的类型，在创建数组时，需要声明数组的长度。下面的代码是使用 Java 语言声明变量。

```
int a=5;
String b="How Are you";
// 数组长度不可变
int[ ] c=new int[ ] {1, 2, 3};
```

在 Python 语言中，可以使用 print( )函数在终端打印内容。

```
my_name="Tony"
print(my_name)
# 终端会输出 Tony
```

可以发现，在 Python 语言中不会显式声明创建的变量的类型，甚至在定义函数时也不需要规定变量的输入类型。Python 解释器会自动对数据的类型进行解析。

例如，数组 c=[1，2，"hello"，4，5]，若要循环打印其中的元素，即 print(c)，Python 解释器会依次解析每个元素的数据类型。因为 1 和 hello 属于不同的数据类型，占用的内存空间是不一样的，而顺序表这种数据结构要求每个元素占用的内存空间相同，因此不能用顺序表来存储 c，所以打印 c 时要依次解析每个元素的数据类型，而不能直接打印。

2. 关键字

在 Python 语言中，关键字是有特殊含义的，这些关键字不能用作函数名或变量名，如表 3-1 所示。

常见关键字

表 3-1　Python 语言的关键字

and	as	assert	async	await
break	class	continue	def	del
elif	else	except	finally	for
from	global	if	import	in
is	lambda	None	nonlocal	not
or	pass	raise	return	try
while	with	yield	FALSE	TRUE

Python 语言的关键字有自己的功能，比如，if、else 关键字用于流程控制，def、return 关键字用于定义函数。

3. 输出变量

print( )函数通常是在 Python 语言中接触到的第一个函数。在 Python 程序中，把这种名称加一组圆括号的对象称为方法或函数，它可以有一个或多个输入参数。print( )函数接收一个输入字符串 Hello World，它就把 Hello World 输出在终端。关于方法和函数的概念，3.2.5 节和 3.2.6 节再详细说明。

3.2.2　运算符与标准库

在 Python 语言中使用运算符进行计算。例如，3＋4＋5 的操作，其中 3、4、5 称为操作数，而＋称为运算符。

```
a=3
b=4
c=5
result=a**2+b**2-c**2        # **2 表示 2 次方
print(result)                # 结果为 0
print(a*b)                   # 结果为 12
print(a/c)                   # 结果为 0.6
# 在 Python 语言中，加号可以用于连接字符串
name="Jack"
intro="My name is "+name
print(intro)                 # 结果为 My name is Jack
```

此外，如果进行更加复杂的计算，如计算半径为 2 cm 的圆的面积，又或者计算 log(12)的结果，可以借助 Python 语言的标准库 math。下面使用标准库 math 进行计算，log( )函数的底数默认为 e。

```
# 使用 import 导入标准库 math
import math
# 定义圆的半径
r=2
# 计算圆的面积 pi*r**2
area=math.pi*r**2
# 计算 log(12)
log_12=math.log(12)
```

此外，还有很多 Python 语言的标准库供用户使用，如 os 库，能够遍历文件夹、修改文件。当要解决一些数据科学问题时，需要使用一些其他社区团队开发的库，这些库称为第三方库，以数据处理库 Pandas 为例，需要在终端安装 Pandas 库后，才能在 Python 程序中使用 Pandas 库的功能。

安装方法是打开 Anaconda Prompt 终端，使用 pip 命令安装 Pandas 库，安装命令如下：

```
pip install pandas
```

3.2.3　流程控制与循环结构

1. 流程控制

Python 语言中的条件语句是通过一条或多条语句的执行结果(True 或者 False)来决定

执行的代码块。流程控制是程序执行的核心功能。

Python 语言中的流程控制一般使用 if、else 、elif(else if 的缩写)。在 Python 语言中，一条控制语句可以包含多个条件，每个条件之间可以使用 and、or 连接。

```
# 判断是否成年
age=16
if age>=18:
    print("成年了")
else:
    print("未成年")
```

上面的代码使用 if、else 判断变量的数值，如果 age 大于或等于 18，打印“成年了”，如果 age 小于 18，打印“未成年”。除了 if、else 语句之外，缩进是 Python 语言中区分代码块的重要工具，一般使用键盘上的 Tab 键或 4 个空格为一个缩进。因此，如果在 if 语句中写一个 for 循环，for 循环中的代码块就需要两个缩进。

```
# 判断成绩
score=85
if score>=90:
   grade='A'
# 在一条判断语句中可以使用多个判断条件
elif score>=80 and score<90:
   grade='B'
elif score>=70 and score<80:
   grade='C'
else:
   grade='D'
```

上面的代码根据成绩得分判断学生成绩属于 A、B、C、D 某一等级。其中使用 if、elif 语句设置多个判断条件，如果所有条件都不满足，可以使用 else 语句打印最终结果。

2. for 循环

Python 语言中的循环语句用于多次执行某一代码块。常见的自然语言处理场景中，可以使用 for 循环读取文本文件中的每一行，处理后再将文本内容保存到新的文本文件中。在视频处理中，可以读取视频中包含的每一帧图像，对图像进行处理。这些应用都需要通过循环功能实现。

```
# 遍历字符串
word="helloworld"
for letter in word:
   print(letter)
```

```
# 对数组中的元素进行遍历
fruits=["apple","banana","peach"]
for fruit in fruits:
    print(fruit)
```

for 循环还可以结合 else 语句使用，for 循环中的语句和普通语句没有区别，else 语句在循环正常执行完的情况下执行，如查找指定数字范围内的质数。

```
for num in range(11, 25):                    # 迭代 11~ 24 之间的数字
    for i in range(2, num):                  # 根据因子迭代
        if num%i==0:                         # 确定第一个因子
            j=num/i                          # 计算第二个因子
            print('%d 等于%d * %d'% (num, i, j))
            break                            # 跳出当前循环
    else:                                    # 循环的 else 部分
        print('%d 是一个质数'% num)
```

3. while 循环

while 循环与 for 循环的功能类似，但在一些场景中 while 循环更加实用。例如，读取一个文本文件，对文本文件逐行处理，可以使用 while 循环读取每一行的内容，判断字符串中内容是否全都为数字，如果全都为数字，则打印该行。

```
raw_file="data.txt"
with open(raw_file, 'r', encoding= 'utf-8')as f_data:
    line=f_data.readline()
    while line:
        if line.strip().isdigit():
            print(line.strip())
        line=f_data.readline()
```

4. break 和 continue

break 语句用来终止循环语句，即 while 循环中条件没有达到 False 或者 for 循环中序列还没被完全遍历完，也会停止执行循环语句。continue 语句用于跳过当前循环语句后续的内容，重新开始新一轮循环。

```
# 当 letter 循环到 Jack 中的 c 时，将不会打印，直接终止循环，结果是 J、a。如果将 break
# 换成 continue，将会打印 J、a、k
for letter in "Jack":
    if letter=='c':
        break
    else:
        print(letter)
```

3.2.4 数据类型

1. 列表

列表是 Python 语言中的重要数据类型，在 JS 语言中被称为数组，Python 语言中的列表具有如下特点：

Python 常见数据类型的区别

(1)列表是 Python 语言中内置的有序、可变序列，列表的所有元素放在一对方括号“[ ]”中，并使用逗号隔开；

(2)当列表元素增加或删除时，列表对象自动扩展或收缩内存，保证元素之间没有缝隙。

在 Python 语言中，一个列表中的数据类型可以各不相同。

```
# 定义列表时，不需要规定列表的长度
list_a=['chinese', 'math', 1997, 2000]    # 列表中可以包含不同类型的数据
list_b=[5, 3, 3, 4, 5]
list_c=["a","b","c","d"]
```

使用下标直接访问列表元素，使用冒号“:”分隔，可以访问列表中的多个元素，如果指定的下标不存在，则抛出异常。

```
list_a=[2, 3, 4, 5, 6]
print(list_a[0])            # 结果为 2，注意列表的下标是从 0 开始索引

# 取下标为 1、2 的两个元素，注意下标为 3 的元素不取，简单记忆为取左不取右
print(list_a[1: 3])         # 结果为[3, 4]
print(list_a[15])           # 访问超出索引值的列表元素会报错
```

错误信息如下：

```
Traceback(most recent call last):
    File"<pyshell#34>", line 1, in< module>
        list_a[15]
IndexError: list index out of range
```

列表的下标从 0 开始计数，如果要访问列表的第一个元素，下标索引是 0；Python 语言中可以使用－1 索引列表的最后一个元素，使用－2 索引倒数第二个元素，以此类推。

使用 append( )方法可以在列表的末尾增加元素。

```
list_a=[3, 4, 5]
list_a.append(6)
print(list_a)          # 结果为[3, 4, 5, 6]
```

使用 insert( )方法可以在列表的指定下标处新增元素。

```
a_List=[3, 4, 5, 7, 9, 11, 13, 15, 17]
a_List.insert(3, 6)     # 在下标为 3 的位置插入元素 6
print(a_List)           # 结果为[3, 4, 5, 6, 7, 9, 11, 13, 15, 17]
```

使用 pop( )方法可以删除元素，默认从列表的末尾删除元素，也可以删除指定下标处的元素，如果指定的下标超出索引则会报错。

```
a_list=[3, 5, 7, 9, 11]
# 使用 pop( )方法删除元素
a_list.pop( )   # 默认从列表结尾删除元素
print(a_list)  # 结果为[3, 5, 7, 9]

a_list.pop(1)  # 删除下标为 1 的元素
print(a_list)  # 结果为[3, 7, 9]
```

2. 元组

元组和列表类似，但元组属于不可变序列，元组一旦创建，用任何方法都不可以修改其元素。

元组的定义方式和列表相同，但定义时所有元素是放在一对圆括号“( )”中，而不是放在方括号中。

```
a_tuple=('a', 'b', 'c', 4, 5, 6)
# 使用 tuple( )函数将其他序列转换为元组
aList=[-1, -4, 6, 7.5, -2.3, 9, -11]
tuple(aList)  # 把列表转换为元组(-1, -4, 6, 7.5, -2.3, 9, -11)
```

元组与列表的区别如下。

(1)元组一旦定义就不允许更改。

(2)元组没有 append( )、insert( )等方法，无法向元组中添加元素。

(3)元组没有 remove( )或 pop( )方法，也无法对元组元素进行 del 操作，不能从元组中删除元素。

(4)从效果上看，tuple( )函数冻结列表，而 list( )函数融化元组。

(5)元组的速度比列表更快。如果定义了一系列常量值，而所需做的仅是对它进行遍历，那么一般使用元组，而不使用列表。

3. 集合

集合是无序、可变序列，使用一对花括号“{ }”界定，集合中的元素不可重复，同一个集合中的每个元素都是唯一的。

集合中只能包含数字、字符串、元组等不可变类型(可哈希)的数据，而不能包含列表、字典、集合等可变类型的数据。

```
a={3, 5}
a.add(7)                                    # 向集合中添加元素
print(a)                                    # 结果为{3, 5, 7}
# 使用 set( )方法将其他类型的数据转换为集合
b_set=set([0, 1, 2, 3, 0, 1, 2, 3, 7, 8])  # 自动去除重复的元素
# 删除元素
a={1, 4, 2, 3}
print(a.pop( ))                             # 结果为 1
```

pop( )方法随机删除集合中的一个元素，并返回删除的元素。

集合支持交集、并集、差集等运算。

```
a_set=set([8, 9, 10, 11, 12, 13])
b_set={0, 1, 2, 3, 7, 8}
a_set| b_set  # 并集, 结果为{0, 1, 2, 3, 7, 8, 9, 10, 11, 12, 13}
a_set.union(b_set)
# 并集的第二种写法, 结果为{0, 1, 2, 3, 7, 8, 9, 10, 11, 12, 13}
a_set & b_set  # 交集, 结果为{8}
a_set.intersection(b_set)
# 交集的第二种写法, 结果为{8}
a_set.difference(b_set)  # 差集, 结果为{9, 10, 11, 12, 13}
a_set - b_set
# 差集的第二种写法, 结果为{9, 10, 11, 12, 13}
```

4. 字典

字典是无序、可变序列。定义字典时，每个元素的键和值用冒号分隔，元素之间用逗号分隔，所有的元素放在一对花括号“{ }”中。

字典中的键可以是任意不可变数据，如整数、实数、复数、字符串、元组等。

1)字典的创建

```
a_dict={'server':'db.diveintoPython3.org', 'database':'mysql'}  # 字典的创建
x={ }        # 空字典
```

2)字典元素的读取

以键作为下标可以读取字典元素，若键不存在则抛出异常。

```
aDict={'name':'Dong', 'sex':'male', 'age':37}
print(aDict['name'])                  # 结果为'Dong'

print(aDict['tel'])                   # 键不存在, 抛出异常
```

错误信息如下：

```
Traceback (most recent call last):
    File"<pyshell# 53>", line 1, in <module>
  aDict['tel']
KeyError: 'tel'
```

使用字典对象的 get( )方法可以获取指定键对应的值，并且可以在键不存在的时候返回指定值。

```
aDict={'name':'Dong', 'sex':'male', 'age':37}
print(aDict.get('address'))          # 结果为 None
print(aDict.get('name'))             # 结果为 Dong
```

使用字典对象的 items( )方法可以返回由字典的键、值组成的元组；使用字典对象的 keys( )方法可以返回字典的键；使用字典对象的 values( )方法可以返回字典的值。

```
aDict={'name':'Dong', 'sex':'male', 'age':37}
for item in aDict.items( ):    # 输出字典中所有元素
    print(item)

# 结果为
#('name', 'Dong')
#('sex', 'male')
#('age', 37)

for key in aDict:    # 不加特殊说明，默认输出键
    print(key)
# 结果为
# name
# sex
# age

for key, value in aDict.items( ):   # 遍历字典
    print(key, value)
# 结果为
# name Dong
# sex male
# age 37
```

3.2.5　函数

1. 标准函数

Python 语言中的标准函数如 len( )、min( )、max( )等，分别可以计算数组长度，返回

数组中元素的最大值和最小值。

```
array_a=[3, 4, 5, 6]
len(array_a)        # array_a 包含 4 个元素，因此 array_a 的长度为 4
min(array_a)        # 结果为 3
max(array_a)        # 结果为 6
```

2. 函数的定义

对于一些更加复杂的功能，需要自行创建函数完成。可以使用 Python 语言中的 def 关键字定义函数。

```
# 定义函数 get_area( )，用于计算长方形面积
# width 和 height 是函数 get_area( )的输入参数，即长方形的宽和高
def get_area(width, height):
    area=width*height
    # 将计算之后的值返回
    return area

# 调用函数
area=get_area(4, 5)
print(area)  # 结果为 20
```

这里的函数和在高中课本上函数的意义基本相同，都是输入一些参数，最终获得一个结果。在 Python 语言中，使用函数的目的是快速组建更复杂的系统。例如，一个函数的功能是对一个无序的序列对象排序，不用关心这个函数到底使用了冒泡排序还是插入排序，重要的是函数能实现排序功能。

函数是组织好的，可重复使用的，用来实现单一或相关联功能的代码段。

函数能提高应用的模块性和代码的重复利用率。Python 语言中由用户自己创建的函数，称为用户自定义函数。

3. 用户自定义函数的特点

(1)函数代码块以 def 关键词开头，后接函数标识符名称和圆括号。

(2)无须定义输入参数的类型。

(3)函数内容以冒号开始，并且缩进。

(4)return[表达式]结束函数，可以返回 0 个、1 个或多个参数。不带表达式的 return 语句相当于返回 None 值。

(5)函数中定义的变量为局部变量，正常情况下在函数外不能使用局部变量。

```
# 定义一个函数，找出列表中能被 7 整除的数，将它们存储到新的列表中并返回
a=[1, 2, 4, 5, 6, 7, 11, 14, 17, 21, 28, 89]
# 使用 def 关键字定义函数
def operation(a_list):
array_a=[ ]
```

```
for a in a_list:
        if a % 7==0:
            array_a.append(a)
        # 返回数组类型的数据
        return array_a
print(array_a) # 会报错，提示无法打印结果，因为 array_a 未被定义，这是因为 array_a 是
# 函数内部的局部变量，无法作为全局变量使用
```

将最后一行语句改为如下两行代码，输出正确结果：

```
result=operation(a)
print(result) # 结果为[7, 14, 21, 28]
```

读者可以试着使用自定义函数，计算给定半径的圆的周长和面积，或者计算一组数据的方差和标准差。

3.2.6　面向对象

1. 面向对象的概念

面向对象是一种抽象化的编程思想。首先介绍以下两个概念：类和对象。

(1)类：描述事物的抽象概念，是对一系列具有相同特征和行为的事物的统称，如动物类。

(2)对象/实例：由类创建出来的真实存在的结果，是类的实例化的结果，如动物类中的鸡、牛、马等。

Python 语言是一门面向对象的语言，本节将对面向对象的特性进行详细介绍。面向对象的目的是对程序开发任务中常见的实体进行归纳总结。例如，一个班级的学生有姓名、年龄、性别等属性，学生有考试、上课等行为，那么单一的学生可以称为对象，而学生本身就是一个类。Python 语言中创建类的代码如下：

```
Class Student:
```

2. 方法与属性

方法是类定义的函数，属性是类中的变量。方法与属性有公有和私有之分，公有方法与公有属性可被外部调用，私有方法与属性一般无法被外部调用，或必须使用特殊的方法调用。下面是一个面向对象的声明方式与实例化方式。

```
class Student:
    def __init__(self, name, gender, age):
        self.name=name
        self.gender=gender
        self.age=age

    def display_student_info(self):
        print(self.name, self.gender, self.age)
```

```
tony=Student("Tony","male","23")
tony.display_student_info()        # 结果为 Tony male 23
```

代码中，第一行声明了类的名称 Student，一般类的名称的首字母大写。对于包含多个单词的类名，使用大写字母开头的驼峰命名法：每个单词的首字母大写。Student 类有两种方法，第一种方法 __init__()是一种特殊的方法，被称为类的构造函数或初始化方法，创建这个类的实例时就会调用该方法，因此后续在实例化 Student 类时，需要按顺序传入 name、gender 和 age 参数的值。第二种方法 display_student_info()是 Student 类的另一种方法，用于打印当前实例包含的变量 name、gender 和 age。除此之外，Student 类中的每种方法都传入了变量 self，self 代指实例化后的实例本身。例如，代码中的 self.age 指的是实例化后 tony 对象的 age 属性。

3. 继承

面向对象的编程带来的主要好处之一是代码的重用，实现这种重用的方法之一是通过继承机制。通过继承创建的新类称为子类或派生类，被继承的类称为基类、父类或超类。

```
class Student:
    def __init__(self, name, gender, age):
        self.name=name
        self.gender=gender
        self.age=age

    def display_info(self):
        print(self.name, self.gender, self.age)

class Teacher(Student):
    def set_teacher_salary(self, salary):
        self.salary=salary

jack=Teacher("Jack","male", 33)
jack.display_info() # 结果为 Jack male 33
jack.set_teacher_salary(5000)
print(jack.salary) # 结果为 5000
```

上面的代码中，Teacher 类继承了 Student 类，因此拥有与 Student 类相同的方法与属性，jack 是 Teacher 类的一个实例，虽然 Teacher 类没有声明 display_info()方法，但是可以直接调用该方法，是因为 Teacher 类继承了 Student 类。此外，Teacher 类可以创建新的方法 set_teacher_salary()。

issubclass()函数用来检查类的继承关系，isinstance()函数用来检查一个实例的类型。下面的代码中 Teacher 类和 Student 类的定义与上面的代码相同。

```
print(issubclass(Teacher, Student))  # 结果为 True
```

3.3 网络爬虫实践

在互联网时代，网络爬虫(简称爬虫)是被普遍运用的一项网络信息搜集技术。该项技术最早被应用于搜索引擎领域，是搜索引擎获取数据的支撑性技术之一。爬虫的目的是收集信息，为数据分析与建模提供数据。例如，一个电影推荐系统想要为用户推荐电影，它首先需要掌握用户对电影的偏好，从电影评分网站上获取各类电影的类型与评分，再基于评分数据建立推荐系统。在构建这个推荐系统的数据获取阶段，如果数据量不大，可以采用人工搜索的方式逐一获取数据，而当数据量增大且数据源不断更新时，就需要使用爬虫工具自动爬取数据。

爬取数据的网站一般带有大量的数据，如旅游网站、电商网站或电影资源网站。以豆瓣电影网站为例，豆瓣电影排行榜如图 3-5 所示，可以通过访问 https://movie.douban.com/chart 获取网站资源。上述链接称为统一资源定位符(uniform resource locator，URL)。

图 3-5　豆瓣电影排行榜

3.3.1　爬虫库与爬虫安全

爬虫可以帮助企业分析数据，制定业务计划。然而，网站会阻止爬虫的爬取活动，因为爬虫需要发送大量的请求，会影响网站服务器的稳定性。在使用爬虫爬取数据时，请遵循以下准则。

(1)不要一次向目标网站发送太多请求，因为这可能会引发危险信号，网站可能会屏蔽客户所在的互联网协议(internet protocol，IP)地址。如果 IP 地址被禁止访问，可以使用轮换 IP 地址并限制某个时间段的请求数量，合法合规地爬取网页内容。

(2)严格禁止通过技术手段绕过服务器的访问限制，或破解被爬网站为保护数据而采取的加密算法及技术保护措施，从而对被爬网站受保护的计算机信息系统中的数据进行爬取。

若被爬网站设定了获取数据信息的措施(包括实名认证、账号密码、内部权限等)，爬虫企业应避免通过伪造实名认证或窃取账号密码、内部权限的形式获取数据。

(3)参考网站中的 robots.txt 文件，合法爬取网站内容。很多网站都会设置 robots.txt 文件，用来规范、约束或者禁止爬虫对网站中数据的采集等操作。robots.txt 文件用于禁止爬虫访问网站的指定目录。robots.txt 文件采用面向行的语法，分为空行、注释行(以 # 开头)、规则行。规则行的格式为 Field:value。常见的规则行包括 User-agent、Disallow、Allow 行。下面是豆瓣网站的 robots.txt。Disallow 表示哪些网站目录中的资源文件是不被允许访问的，如果违反该约束就会被网站封 IP 地址。Allow 表示哪些网站目录中的资源文件是被允许访问的。

```
User-agent:*
Disallow:/subject_search
Disallow:/amazon_search
Disallow:/search
Disallow:/group/search
Disallow:/event/search
Disallow:/celebrities/search
Disallow:/location/drama/search
Disallow:/forum/
Disallow:/new_subject
Disallow:/service/iframe
Disallow:/j/
Disallow:/link2/
Disallow:/recommend/
Disallow:/doubanapp/card
Disallow:/update/topic/
Disallow:/share/
Allow:/ads.txt
Sitemap:https://www.douban.com/sitemap_index.xml
Sitemap:https://www.douban.com/sitemap_updated_index.xml
# Crawl-delay:5
```

3.3.2 爬虫流程

爬虫就是让计算机浏览网页，然后自动把感兴趣的内容以格式化的方式截取出来，并持久化地保存到文本文件或关系型数据库中。一个标准的爬虫流程可以简单地分为以下几个步骤。

(1)获取需要爬取的网页。

(2)爬取网页内容。

(3)解析爬取的内容。

(4)清洗和组织数据。

在爬取网站内容前，请浏览网站的 robots.txt 文件，确认感兴趣的内容可以被爬取，再进行爬虫操作。在使用爬虫爬取网站内容时，标准做法是先使用 Python 语言的 requests 库将网站的内容缓存到本地，requests 库可以伪装成浏览器，将网页访问请求发送给目标服务器，并获得网站的内容。由于网站的内容是半结构化的 HTML 文本，文本中包含了大量的 HTML 标签和 CSS 修饰文本，使用 Python 语言的正规表达式获取感兴趣的内容的难度较高，因此可以使用爬虫工具库 BeautifulSoup 解析缓存到本地的网页内容，把页面解析成较容易处理的对象，然后使用 BeautifulSoup 库内置的方法从中找到感兴趣的数据，并一一提取出来，然后将数据保存到文本文件或数据库中。

3.3.3　案例介绍

本节案例使用豆瓣电影排行榜的数据实现一个简单的爬虫应用，从豆瓣电影排行榜爬取排名靠前电影的名称、评分和电影介绍网页 URL。需要使用到的库有 requests 和 BeautifulSoup。

requests 库是一个用于发送超文本传送协议(hypertext transfer protocol，HTTP)请求的第三方库，它提供了一种简洁而易用的方式与 Web 服务通信，用于获取网页内容、发送 POST 请求、处理 Cookie 和 Session 等。BeautifulSoup 库是一个用于解析 HTML 和 XML 文档的第三方库，它提供了一种简单而灵活的方式来从网页中提取数据。除了 BeautifulSoup 库，还有一些与之类似的第三方库(如 lxml、pyquery、html5lib 等)可以解析 HTML 和 XML 文档，提取数据和处理网页内容。

首次使用 requests 库和 BeautifulSoup 库前，需要使用 pip 命令安装库，再导入库。

通过浏览器访问豆瓣电影排行榜，可以看到高评分电影列表如图 3-6 所示，其中，一个方框标记的电影名称及其链接 URL，另一个方框标记的电影评分是本次需要爬取的内容。

图 3-6　豆瓣电影排行榜

使用 requests 库可以获取当前页面的内容，代码如下所示。其中前两行使用 import 语句导入爬虫需要用到的库，然后定义了 headers 参数，用于保存访问网站时的请求头信息。get_url( )函数使用 GET 请求方法从服务器获取网站页面的所有内容。

```
import requests
from bs4 import BeautifulSoup
headers={
    'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
(KHTML, like Gecko)'
    'Chrome/93.0.4577.63 Safari/537.36 Edg/93.0.961.47'
}

def get_url(url):
    response=requests.get(url=url, headers=headers)
    if response.status_code==200:
        return response.text
    else:
        print(f'状态码错误...{response}')
```

get _url( )函数返回的内容是 response.text，是字符串类型(str)，将获取的内容打印出来，部分内容如下：

```
<! DOCTYPE HTML>
<HTML lang="zh-CN" class="ua-windows ua-webkit">
<head>
    <meta http-equiv="Content-Type" content="text/HTML; charset=utf-8">
    <meta name="renderer" content="webkit">
    <meta name="referrer" content="always">
    <meta name="google-site-verification"
    content="ok0wCgT20tBBgo9_zat2iAcimtN4Ftf5ccsh092Xeyw" />
    <title>
          豆瓣电影排行榜
    </title>
    <meta name="baidu-site-verification" content="cZdR4xxR7RxmM4zE" />
    <meta http-equiv="Pragma" content="no-cache">
    <meta http-equiv="Expires" content="Sun, 6 Mar 2005 01:00:00 GMT">
```

可以看到文本内容是半结构化的 HTML 文档，和在浏览器中看到的内容差别较大，这是因为 HTML 文本需要经过标签定义与 CSS 代码渲染，才能最终变成浏览器中的内容。为了获取需要采集的数据，可以对文本内容进行搜索，一个可行的方案是使用 Python 语言自带的 re 库，用正规表达式从文本中提取每一部电影的名称、评分等，这种方法需要撰写较为复杂的正规表达式，对于初学者来说较为困难。对于 HTML 结构的文本，可以使用

lxml 或 BeautifulSoup 等库解析 HTML 结构的文本并使用它们定位 HTML 文本中的节点信息，快速获取感兴趣的数据。使用 BeautifulSoup 库解析 HTML 文本的代码如下：

```
url="https://movie.douban.com/chart"
content=get_url(url)
soup=BeautifulSoup(content, 'html.parser')
```

其中，url 变量是豆瓣电影排行榜页面的地址，get_url( )函数会根据传入的网址，将访问到的信息缓存到本地，然后根据 content 变量，对豆瓣电影排行榜页面的 HTML 文本进行解析，实例化为一个 BeautifulSoup 类的对象，以便于后续采集数据。

为了获取电影名称、评分和电影介绍网页 URL，需要定位这些数据所在的位置，即观察 div 标签位置与 class 属性的内容。通过这些位置信息，使用 BeautifulSoup 库获取需要的数据。直接观察缓存到本地的文本信息不容易获取电影名称所在的位置，可以使用浏览器自带的调试工具提取位置信息。在 Chrome 内核的浏览器中，在网页上右击并在弹出的快捷菜单中选择“检查”命令，或直接按 F12 键，就可以进入调试者模式，如图 3-7 所示，单击调试工具窗口中的 elements 按钮，会显示当前网页的 HTML 代码。

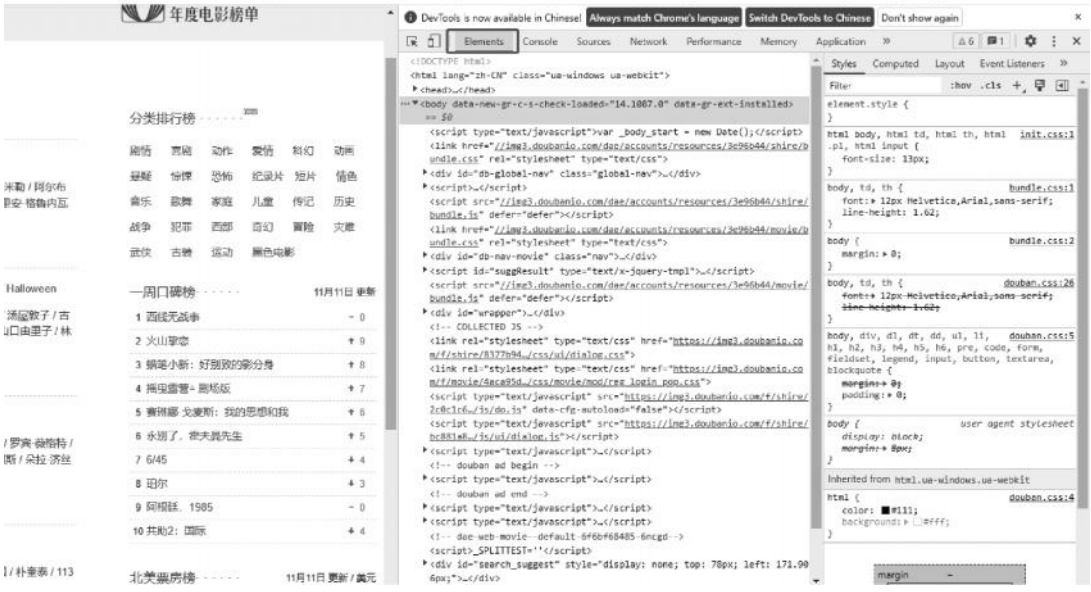

图 3-7　调试者模式

在调试工具窗口中，找到左上角的按钮，单击该按钮，然后将鼠标指针放在网页上某一位置，鼠标指针当前所在位置的 div 块会高亮，调试工具窗口的 HTML 代码中对应的 div 标签也会高亮，如图 3-8 所示。可以看到，当鼠标指针放在电影简介上时，其对应的 div 标签的 class 属性为 pl2，因此可以在网页的 HTML 文本中查找这类 div 标签，从而获取每一部电影的详细信息。随着鼠标指针的移动，可以对网页内的所有内容进行搜索，并获取对应的标签。

图 3-8　使用调试工具获取网页内容

根据用调试工具获取的标签信息，使用 BeautifulSoup 库提供的工具获取指定信息，代码如下所示：

```
    movies_intro=soup.find_all(name='div', attrs={"class":"pl2"})
    for movie_intro in movies_intro:
        title=movie_intro.find_all("a")
        rating_content=movie_intro.find_all(name="div", attrs={"class":"star clearfix"})
        if len(rating_content) ==0:
            continue
        rating_score=rating_content[0].find_all(name="span", attrs={"class":"rating_nums"})
        rating=rating_score[0].text
        movie_intro_url=title[0].get('href')
        movie_title_raw=title[0].text
        movie_title=movie_title_raw.split('/')[0].strip()
        print(movie_title+"\t"+rating+"\t"+movie_intro_url)
```

soup.find_all()方法用于获取 class 属性为 pl2 的 dir 标签的代码块。对返回的结果进行打印，部分内容如下：

```
    [<div class="pl2">
     <a class=""href="https://movie.douban.com/subject/3042261/">
                西线无战事
                    /<span style="font-size:13px;">新西线无战事/All Quiet on the Western Front</span>
    </a>
```

```
    <p class="pl">2022-09-12(多伦多电影节)/ 2022-09-29(德国)/2022-10-28(德国网络)/费利克斯·卡米勒/阿尔布雷希特·舒赫/亚伦·希尔默/丹尼尔·布鲁赫/塞巴斯蒂安·胡克/莫里茨·克劳斯/阿德里安·格鲁内瓦尔德/埃丁·哈萨诺维奇/蒂博·德·蒙塔朗贝尔...</p>
    <div class="star clearfix">
    <span class="allstar45"></span>
    <span class="rating_nums">8.6</span>
    <span class="pl">(64832 人评价)</span>
    </div>
    </div>,
    <div class="pl2">
    <a class="" href="https://movie.douban.com/subject/35437938/">
                        名侦探柯南:万圣节的新娘
                      /<span style="font- size:13px;">名侦探柯南剧场版 25/Detective Conan:The Bride of Halloween</span>
    </a>
    ...
```

find _all( )方法会找到所有满足要求的 div 标签，并将它们存到一个列表中，列表中的每个元素对应一个满足要求的 div 标签。例如，要获取第一部电影的 HTML 文本，则需要访问 movies _intro[0]。

接下来需要逐个处理每一部电影的 HTML 文本，获取感兴趣的内容。for 循环中的两个 find _all( )方法分别用于获取 a 标签(含标题和链接 URL)和 class 属性为 rating _nums 的 span 标签(含评分)。然后，rating 变量表示电影评分，movie _intro _url 变量表示电影介绍网页 URL，movie _title 变量表示电影名称。最后打印的解析结果如下：

```
    西线无战事     8.6 https://movie.douban.com/subject/3042261/
    名侦探柯南：万圣节的新娘     7.9 https://movie.douban.com/subject/35437938/
    危笑     6.3 https://movie.douban.com/subject/35779959/
    6/45     7.7 https://movie.douban.com/subject/35441582/
    共助 2：国际     7.1 https://movie.douban.com/subject/35240874/
    黑亚当     6.0 https://movie.douban.com/subject/26959697/
    火山挚恋     9.0 https://movie.douban.com/subject/35694766/
    福尔摩斯小姐：伦敦厄运     6.6 https://movie.douban.com/subject/35461749/
    珀尔     7.3 https://movie.douban.com/subject/35801819/
    新神榜：杨戬     7.0 https://movie.douban.com/subject/35360684/
```

习　　题

1. 写出下列代码的输出结果。

```
for letter in "Jack":
    if letter=='c':
       continue
    else:
        print(letter)
```

2. 写出下列代码的输出结果。

```
list01=['python', 786, 2.23, 'john', 70.2]
print(list01[0])
print(list01[-1])
print(list01[0:3])
```

3. 简述 Python 语言中元组、列表、集合与字典的区别。如果想改变元组中的元素，可以采用什么方式？

4. 写出下列代码的输出结果，并观察其中有什么规律。

```
num_list=[ ]
for num in range(2, 100):
    for i in range(2, num):
            if num% i==0:
            break
    else:
        num_list.append(num)
print(num_list)
```

5. 使用 Python 语言中的 while 循环与 break 关键字实现一个数值猜大小游戏。程序接收数值为输入，若输入的数值大于设定值，则打印“大了”；若输入的数值小于设定值，则打印“小了”；如果输入的数值与设定值相同，则打印“猜对了”，并且程序终止。

习题解析